

# Gnome Sort

35 30 4 27 2 10  
↑ ↑

30 35 4 27 2 10  
↑

30 4 35 27 2 10  
↑

4 30 35 27 2 10  
↑

4 30 27 35 2 10

4                      30                      27                      35                      2                      10

↑

[illegible][illegible]

4      27      2      30      35      10

↑

4                  2      27                  30                  33      10

↑

[illegible]

2                  4                  27                  30                  10                  35

↑

2            4        27          10            30            35

↑

2 4 10 27 30 35

- Posso camponde com interior
- Se for menor trees e volta
- um posição

# Comb Sort

↳ Subor bubble sort

↳ Problems do bubble: Números pequenos no final o algoritmo fica lento.

↳ Solução: Em vez de eu comparar os cores perto, porque não usar um gap maior.

$(arr[i], arr[i+1]) \Rightarrow (arr[i], arr[i+gap])$

$$g \circ p = 6$$

30 5 2 22 1 15 3

↑

Op = 4

3      5      2      2 2      1      15      30  
↑      ↑      ↑           ↑           ↑

$$g^o p = 3$$

1      5      2      2 2      3      15      30

↑      ↑      ↑      ↑

$$\text{gap} = 2$$

1      3      2      22      5      15      30

↑      ↑      ↑      ↑      ↑

$$\text{gap} = 1$$

1      3      2      15      5      22      30

↑      ↑

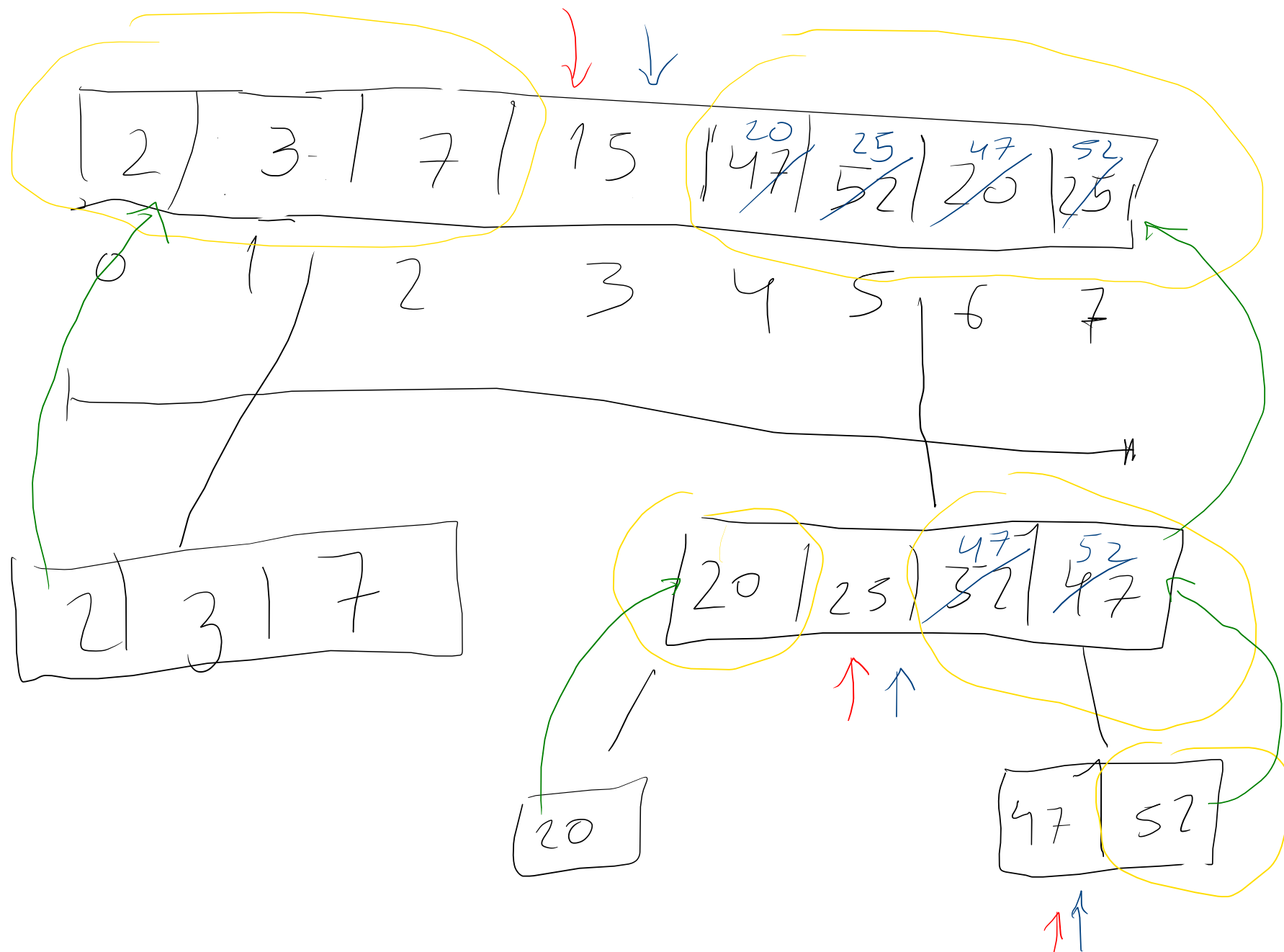
1      2      3      5      15      22      30

· Inicializo  $\text{gap} = n$

· Posso em cada posição do array, comparar  $i$  e  $i + \text{gap}$ . Se for maior troco.

· Atualizo  $\text{gap} = \text{gap} / 1.3$ , até  $\text{gap} = 0$ .

# Quick Sort

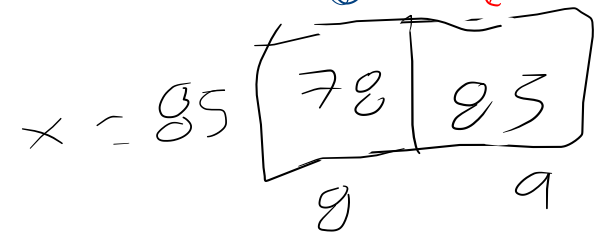
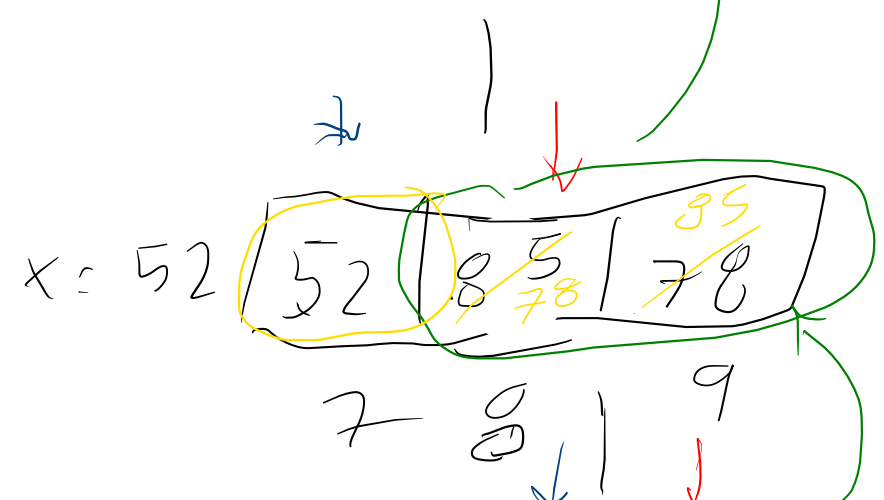
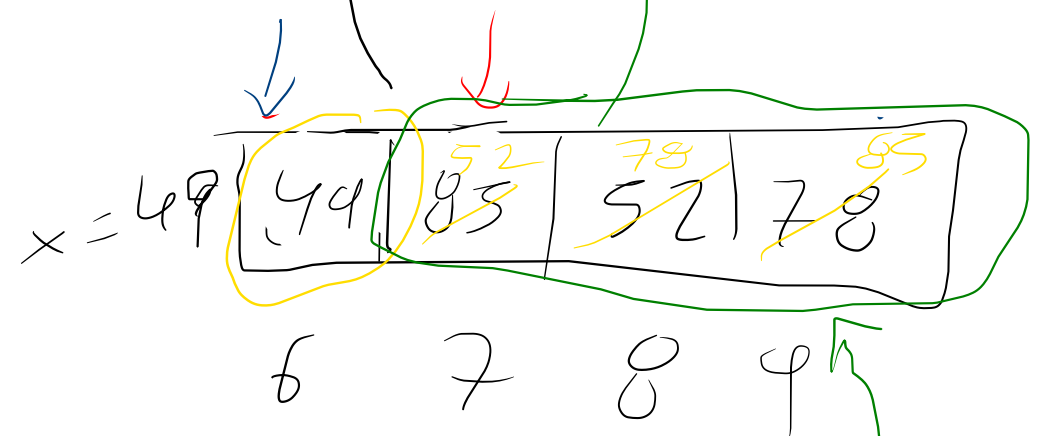
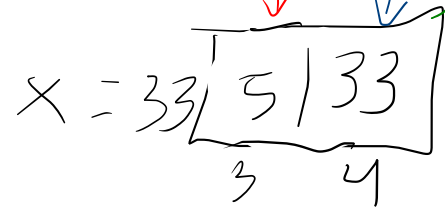
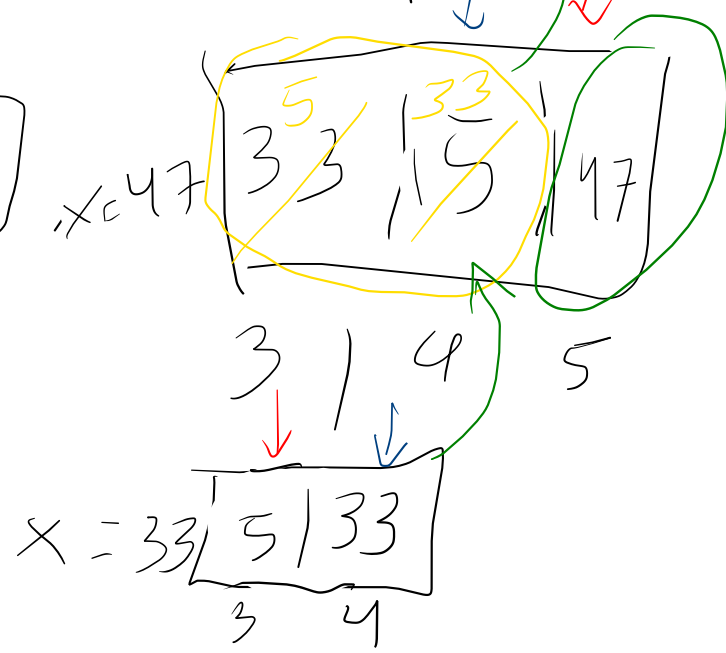
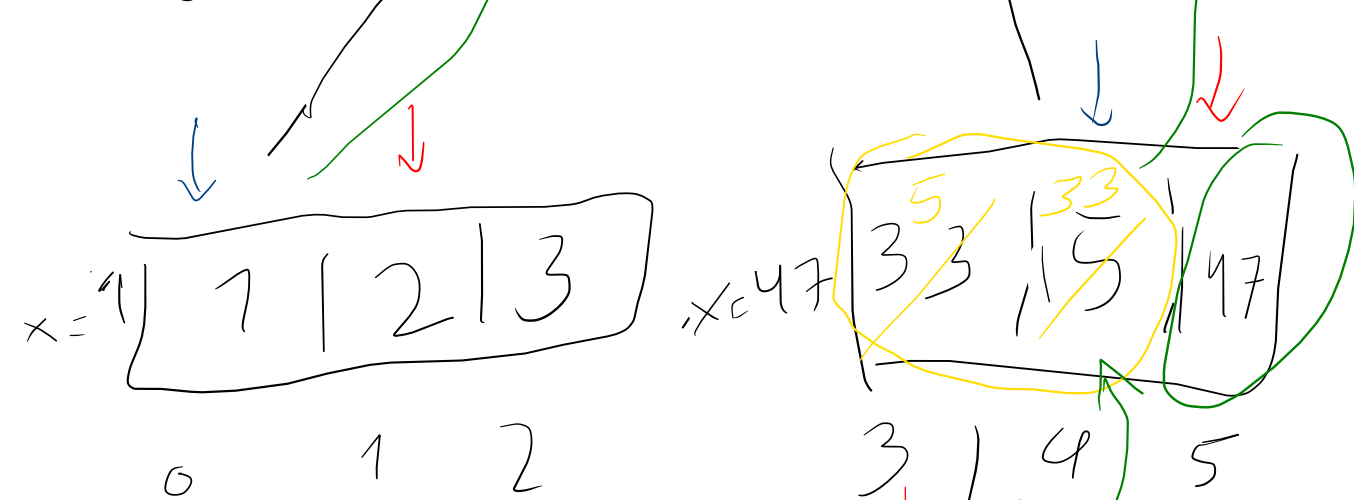
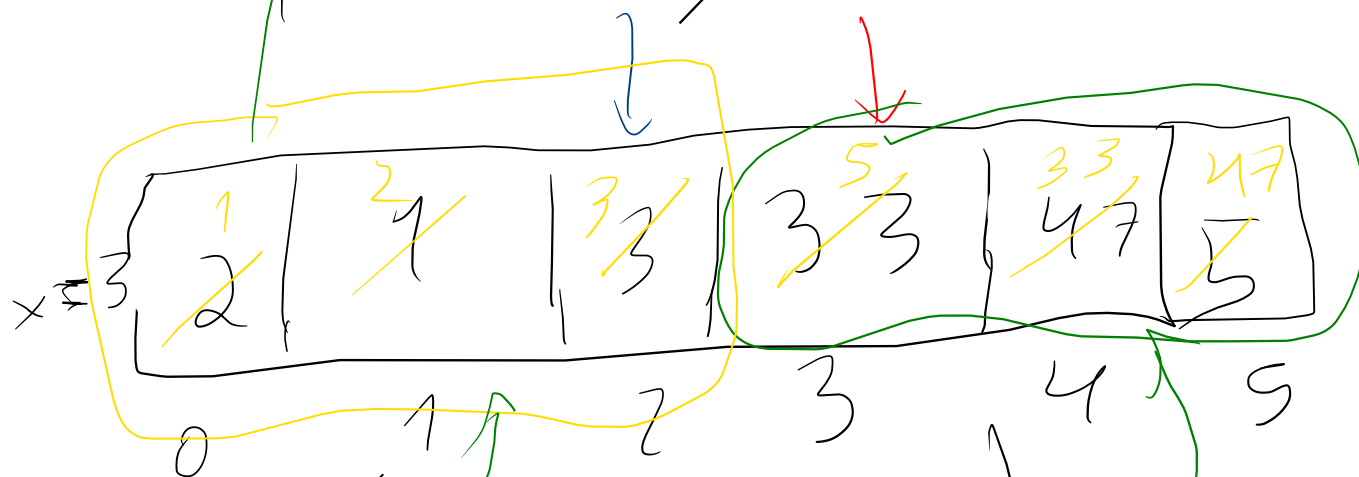
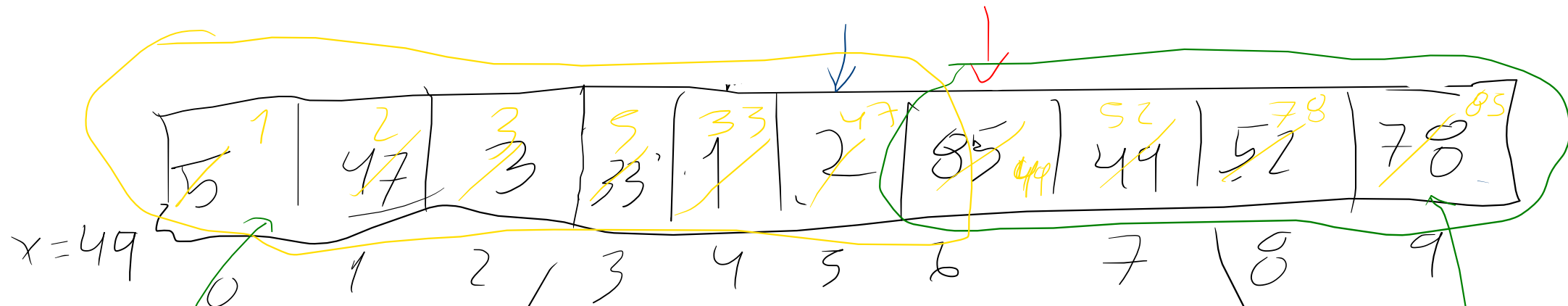


· Pega o intervalo até ao menor  
e acha o ponto (divide  
o intervalo) e faz o mesmo  
coisa para cada intervalo recursi-  
vamente.

· Partição vamos passar por  
os elementos comparando os  
elementos do começo com o  
fim, caso eles não respeitem a ordem  
troque-os.



# Quick Sort com Pivo



Shell Sort

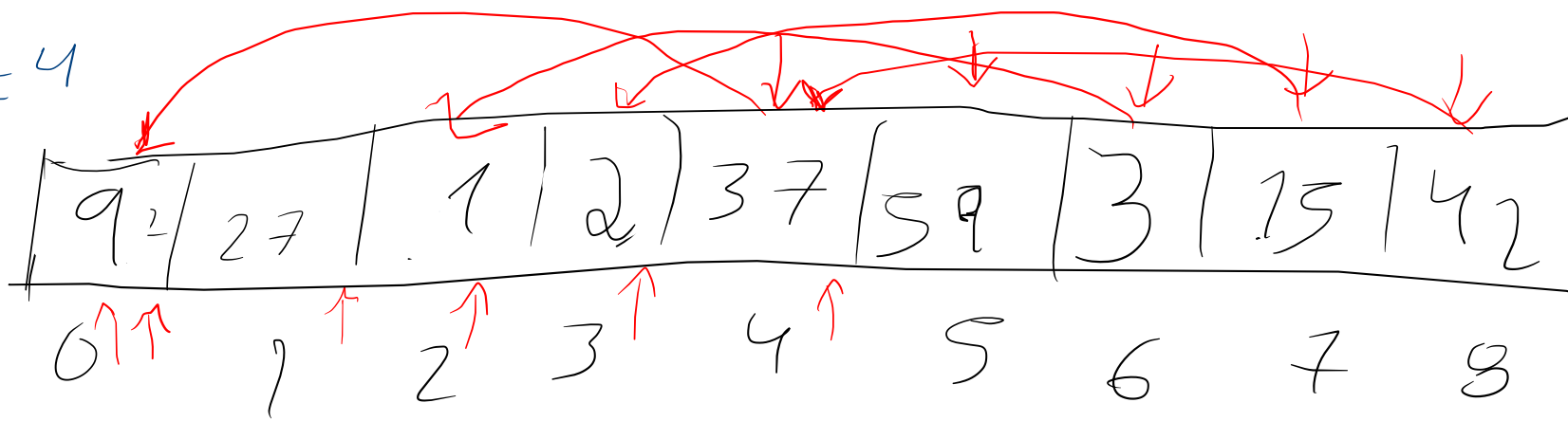
↳ Insertion Sort can gap.

↳ Knuth:  $n = 3 * h + 1$

$$n = 9$$

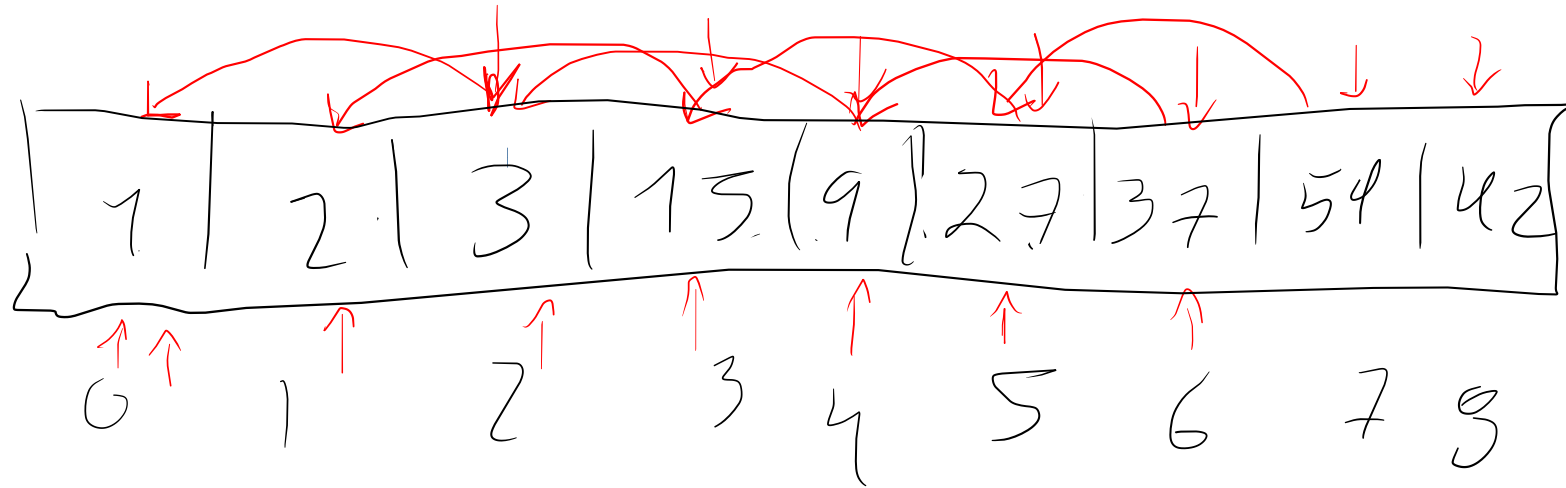
$$n = 1 \rightarrow 4 \rightarrow 13 / 3 = \textcircled{4} \rightarrow \text{gap}$$

$$h=4$$



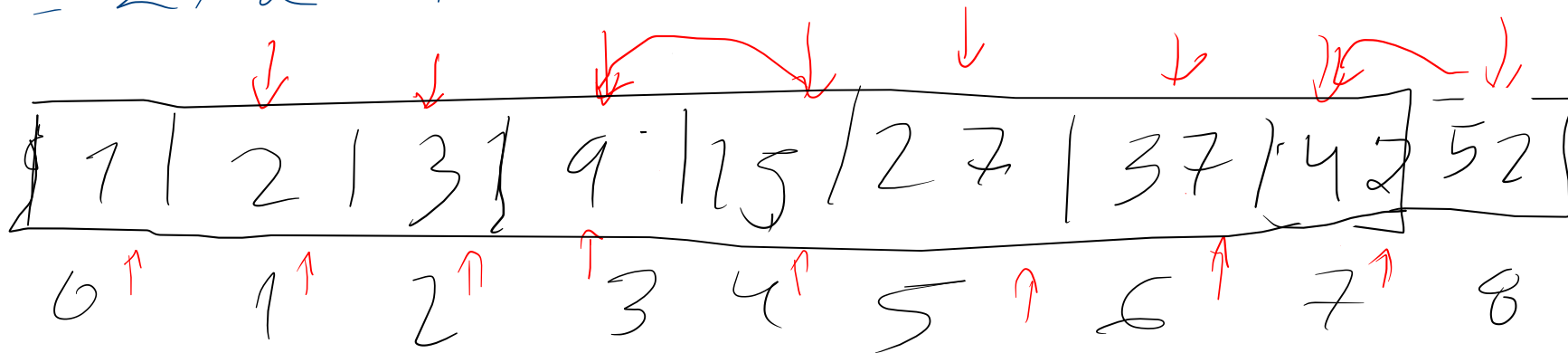
~~42~~, ~~31~~, ~~1~~  
~~2~~, ~~9~~

$$h=4/2=2$$



~~1~~, ~~2~~, ~~3~~,  
~~15~~,

$$h=2/2=1$$

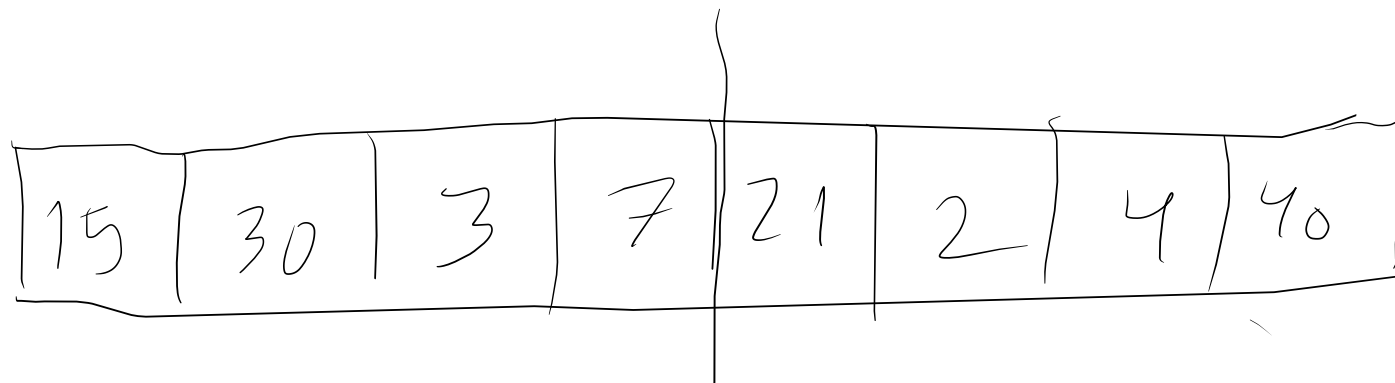


~~1~~, 42

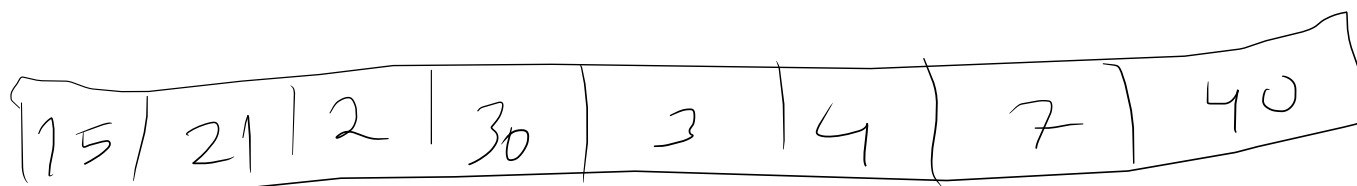
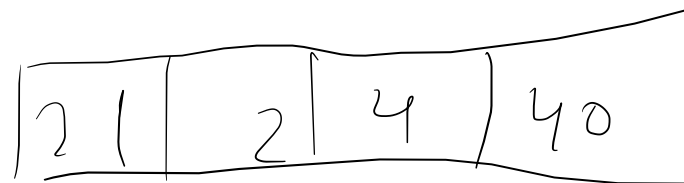
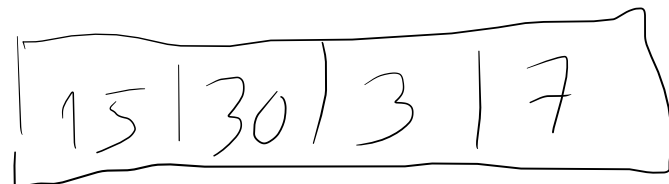
Merge Sort

↳ só funciona para potência de 2:

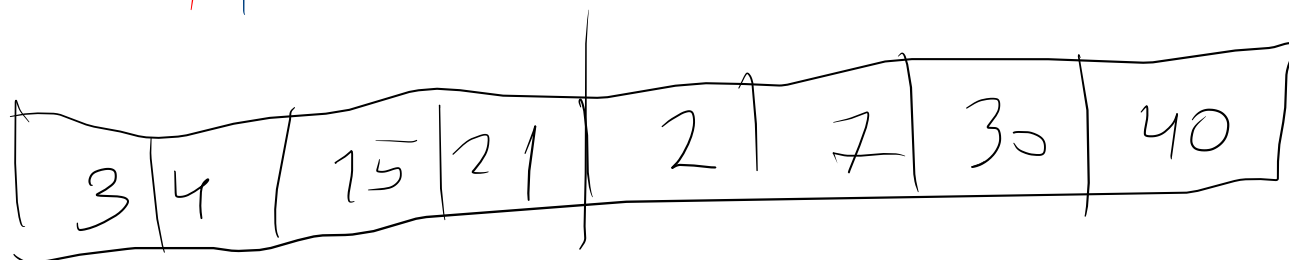
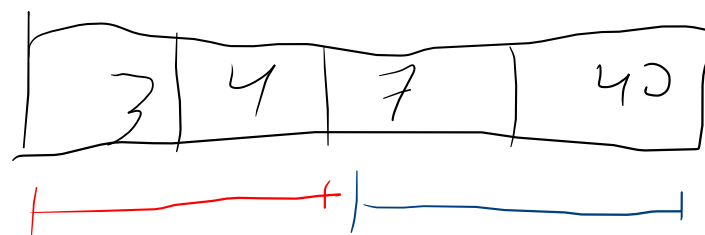
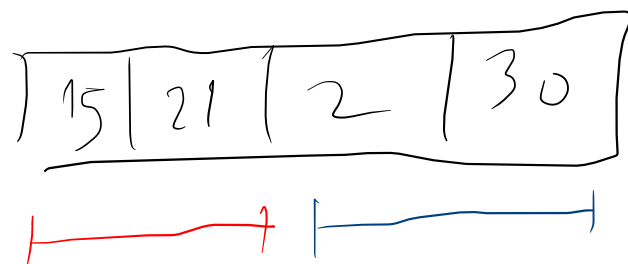
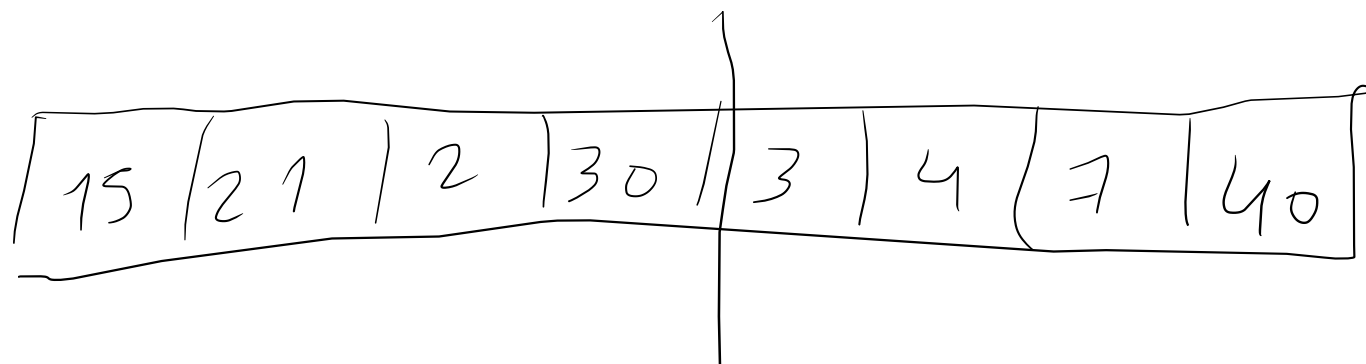
2, 4, 8, 16, ...,  $2^n$



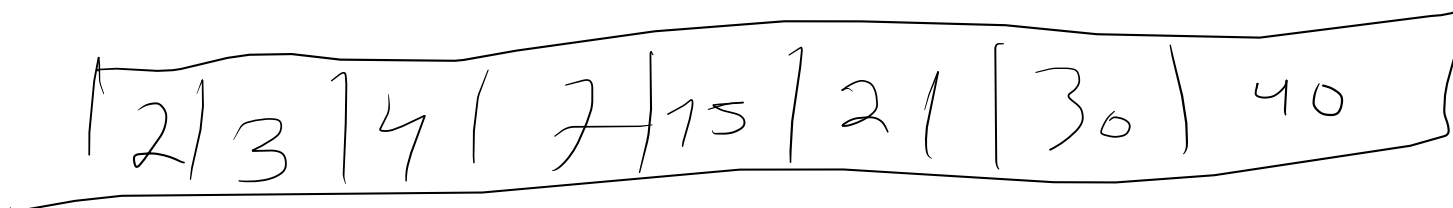
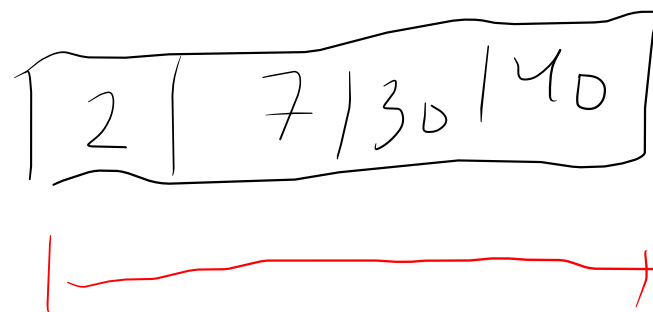
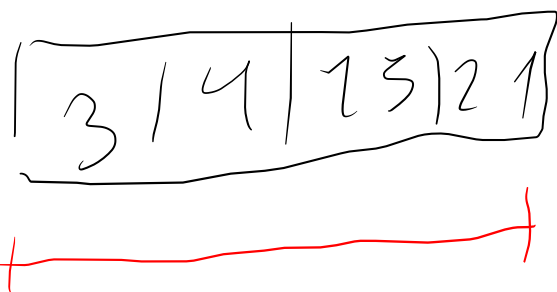
• seq = 1



$$seq = 2$$

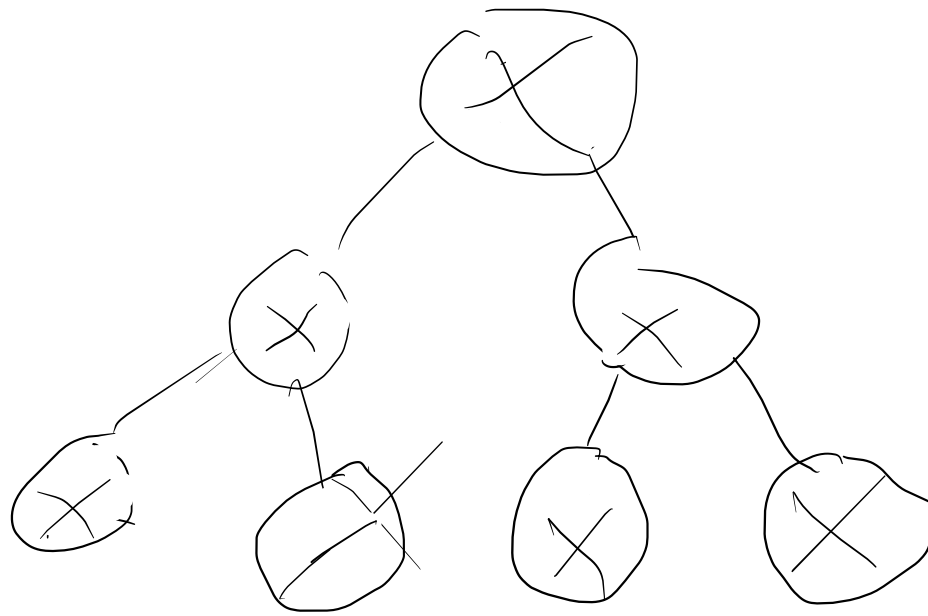


$$seq = 4$$



# Heap Sort

|   |    |    |    |    |   |   |
|---|----|----|----|----|---|---|
| 3 | 15 | 30 | 12 | 21 | 4 | 2 |
|---|----|----|----|----|---|---|



|   |   |   |    |    |    |    |
|---|---|---|----|----|----|----|
| 2 | 3 | 4 | 12 | 15 | 21 | 30 |
|---|---|---|----|----|----|----|

|   |   |   |    |    |    |    |
|---|---|---|----|----|----|----|
| 2 | 3 | 4 | 12 | 15 | 21 | 30 |
|---|---|---|----|----|----|----|

$$n = 4$$

$$poi = 4/2 - 1 = 1 = 0$$

$$n = 3$$

$$poi = 3/2 - 1 = 0$$

$$n = 2$$

$$poi = 2/2 - 1 = 0$$

$$n = 7$$

$$poi = 7/2 - 1 = 2$$

$$e = 2 * poi + 1$$

$$d = 2 * poi + 2 = e + 1$$

$$poi = 1, p0i = 0$$

$$n = 6$$

$$poi = 6/2 - 1 = 2$$

$$poi = 1, p0i = 0$$

$$n = 5$$

$$poi = 1 = 0$$

# Tim Sort

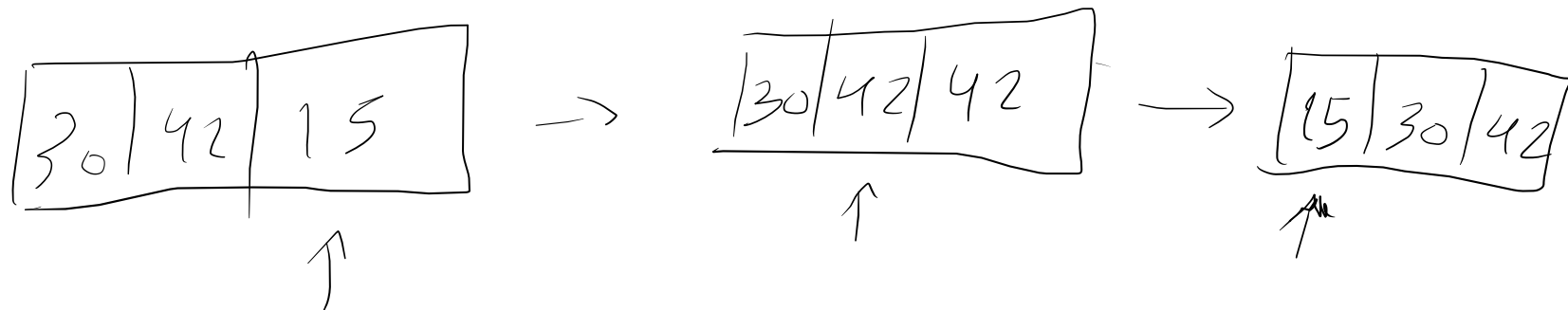
↳ Insertion Sort + Merge Sort

L) Voumo aproveitar os pedzcos ordenados (pvlb)

L) Run: Pedos 5 ordenados (3)

|    |    |    |   |    |    |   |   |    |    |    |    |    |    |
|----|----|----|---|----|----|---|---|----|----|----|----|----|----|
| 15 | 30 | 42 | 2 | 17 | 12 | 1 | 3 | 22 | 34 | 60 | 61 | 1  | 2  |
| 0  | 1  | 2  | 3 | 4  | 5  | 6 | 7 | 8  | 9  | 10 | 11 | 12 | 13 |

$$o_{rr} \vdash J = \{ \{0, 2\}, \{3, 5\}, \{6, 11\}, \{12, 7\} \}$$





|    |    |    |   |    |    |   |   |    |    |    |    |    |    |
|----|----|----|---|----|----|---|---|----|----|----|----|----|----|
| 15 | 30 | 42 | 2 | 11 | 12 | 1 | 3 | 22 | 34 | 60 | 61 | 1  | 2  |
| 0  | 1  | 2  | 3 | 4  | 5  | 6 | 7 | 8  | 9  | 10 | 11 | 12 | 13 |

$$orr [ ] = \{ \{ \underline{0}, \underline{2} \}, \{ \underline{3}, \underline{5} \}, \{ \underline{6}, \underline{11} \}, \{ \underline{12}, \underline{13} \} \}$$

|   |    |    |    |    |    |   |   |    |    |    |    |   |   |
|---|----|----|----|----|----|---|---|----|----|----|----|---|---|
| 2 | 11 | 12 | 15 | 30 | 42 | 1 | 3 | 22 | 34 | 60 | 61 | 1 | 2 |
|---|----|----|----|----|----|---|---|----|----|----|----|---|---|

|   |   |   |   |    |    |    |    |   |    |    |    |    |    |   |   |   |   |    |    |    |    |
|---|---|---|---|----|----|----|----|---|----|----|----|----|----|---|---|---|---|----|----|----|----|
| 1 | 1 | 2 | 3 | 22 | 34 | 60 | 61 | 2 | 11 | 12 | 15 | 30 | 41 | 1 | 1 | 2 | 3 | 22 | 34 | 60 | 61 |
|---|---|---|---|----|----|----|----|---|----|----|----|----|----|---|---|---|---|----|----|----|----|

$$orr [ ] = \{ \{ 0, 5 \}, \{ 6, 13 \} \}$$

|   |   |   |   |   |    |    |    |    |    |    |    |    |    |
|---|---|---|---|---|----|----|----|----|----|----|----|----|----|
| 1 | 1 | 2 | 2 | 3 | 11 | 12 | 15 | 22 | 30 | 34 | 41 | 60 | 61 |
|---|---|---|---|---|----|----|----|----|----|----|----|----|----|